Adding Delaunay Triangulation to the k-Center Project

Progress note for the next review: what the triangulation buys us, results on first-center selection and outlier removal, measured time complexities, and what to build next.

Where the project stands

The core is unchanged and still correct: Gonzalez's greedy farthest-point algorithm, O(nk), with its proven 2× guarantee. What is new is that the project now has a **geometric spine**. Following the suggestion to bring in computational geometry, every added feature is derived from a single Delaunay triangulation rather than from unrelated bolt-on heuristics.

That matters for more than tidiness. The triangulation is built once in O(n log n) and then answers three separate questions that would otherwise each need their own $O(n^2)$ pass. The project's claim is no longer "we tried some seeding rules" but "one structure, three uses, each with a measured speed-up".

Question the project asks	Without Delaunay	With Delaunay	Measured
Which point should be the first center?	`O(n²) dense MST + all-pairs`	`O(n log n) MST from Delaunay edges + double sweep`	10.3× faster, identical answer
Which points are outliers?	`O(n²) pairwise, or a separate KD-tree`	`O(n) — the 1-NN distance is already an edge of the mesh`	F1 0.89 vs 0.63
Where is the farthest uncovered spot?	`O(res²·k) grid scan, approximate`	`O(k log k) — Voronoi vertices are Delaunay circumcentres, exact`	619× fewer candidates, 55× faster

Why the MST speed-up is legal, and not a trick: the Euclidean minimum spanning tree is a **subgraph** of the Delaunay triangulation (Shamos & Hoey, 1975). So Kruskal only ever has to consider the roughly 3n Delaunay edges, never all n(n–1)/2 pairs. The answer is provably the same tree.

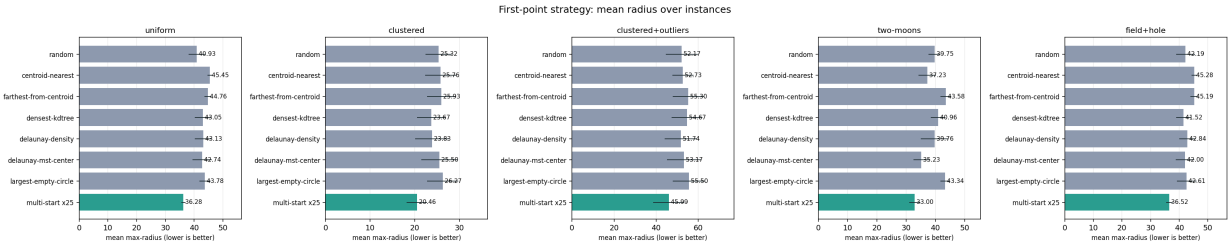
Why the Voronoi step is exact, not sampled: the Voronoi diagram is the straight-line dual of the Delaunay triangulation, and every Voronoi vertex is the circumcentre of some Delaunay triangle. The largest-empty-circle problem that Gonzalez implicitly solves at each step therefore has a finite, exact candidate set.

Focus 1 — Which point should be the first center?

The 2× proof holds no matter which point you start from, so in theory the choice is free. On a specific dataset it is not free at all. We ran seven seeding rules on five dataset families, k = 6, n = 400, twelve freshly generated instances each, every rule seeing the same instances.

> *Result, stated plainly:* ***the first point matters a great deal, and no cheap deterministic rule reliably exploits that.***

Finding	Evidence
The starting point changes the achieved radius substantially	Within a single instance, 25 random starts span a 25–40% range of final radii
No deterministic rule beats a random start on a majority of datasets	Best deterministic rules win on only 2 of 5 dataset families
Multi-start dominates everything	`Best on 5 of 5 families, at O(mnk) — 25× the work for roughly 10–20% smaller radius`
The one place geometry clearly wins is non-convex data	On two-moons, the Delaunay MST centre is the best deterministic rule (1.08× vs 1.12× for centroid-nearest)
Computing the MST-centre seed via Delaunay is 10× cheaper	`19.8 ms dense O(n²) vs 1.9 ms Delaunay O(n log n) at n = 400, same chosen point`



Mean achieved radius per seeding rule across twelve instances per dataset family; whiskers are one standard deviation. Green marks the best rule for that family. The error bars are the point: most gaps between deterministic rules are smaller than the run-to-run spread, which is why the honest conclusion is a negative one.

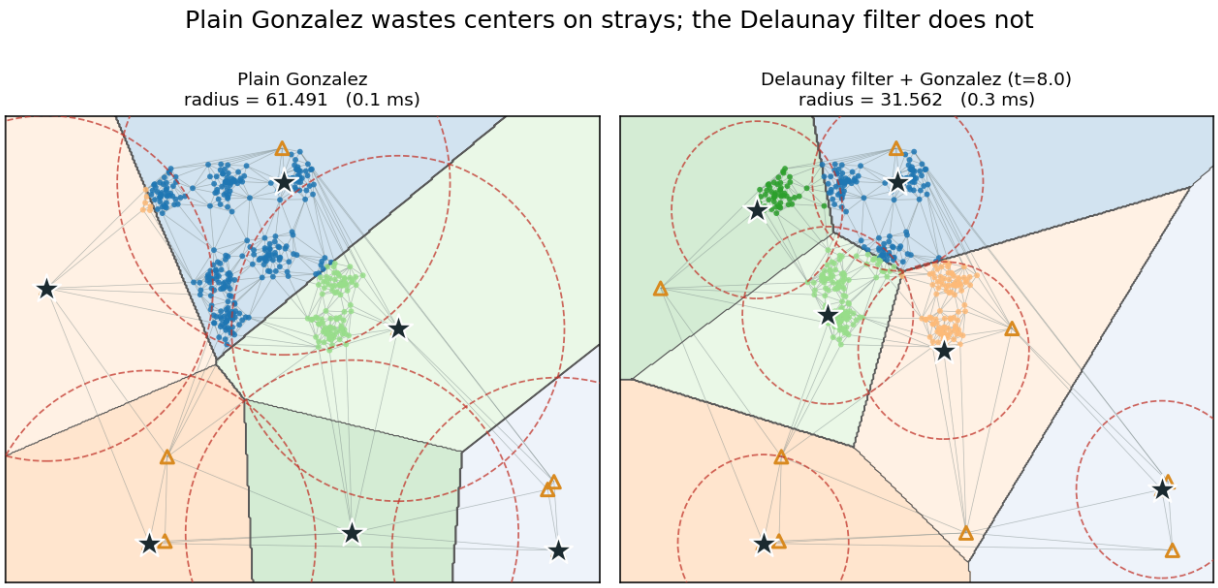
Why this is a good result, not a failed one

A negative result that is properly measured is worth more in a review than a positive one that is not. Reporting "our clever seeding rule wins" on a single instance would not survive a question about variance. Reporting "the choice matters, here is the spread, and here is the one geometric case where structure helps" is defensible and points at the next experiment.

It also matches the theory rather than contradicting it. The 2× bound is start-independent, so there was never a reason to expect a universal best seed. What the data adds is the size of the practical gap and where geometry closes it.

Focus 2 — Removing outliers

Gonzalez's rule takes the single farthest point, so one stray coordinate becomes a center. With k = 6 and ten planted strays, plain Gonzalez spends most of its budget on empty ground.



Left: plain Gonzalez, radius 61.5, with most centers stranded on strays (amber diamonds). Right: the same data after the Delaunay filter, radius 31.6. Grey hairlines are the Delaunay edges; shaded areas are the Voronoi coverage regions.

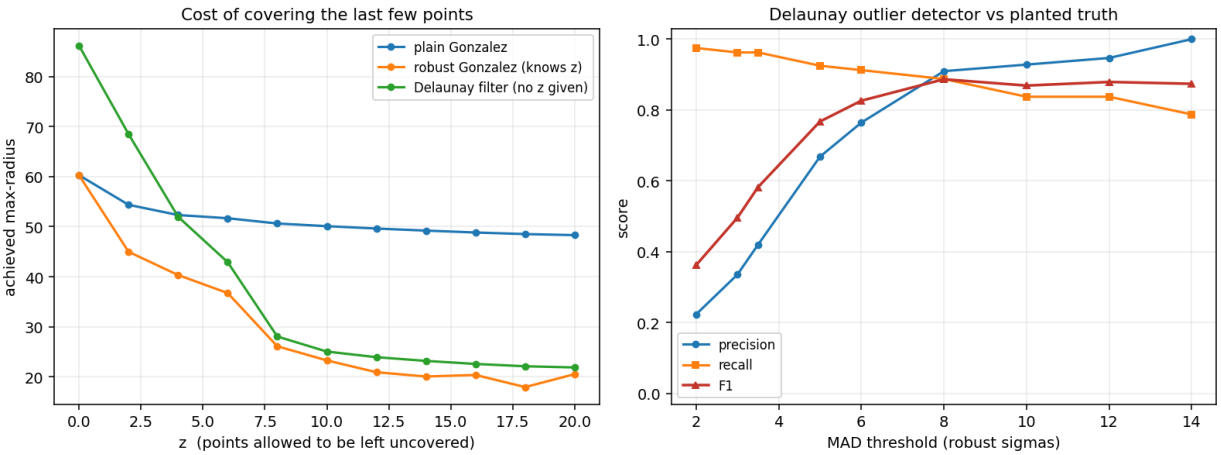
The detector, and the one design decision that mattered

The first attempt scored each point by the **mean** length of its incident Delaunay edges — the obvious density measure. It worked badly: F1 peaked at 0.63. The reason is worth putting on a slide, because it is a geometry argument, not a tuning accident.

> *A point on the **rim** of a cluster has some very long edges reaching across the gap to the next cluster, so its mean edge length is large and it gets flagged as an outlier. But it still has close neighbours inside its own cluster. A genuinely isolated point has no close neighbour at all.*

So the score was changed to the point's **minimum** incident edge length. That value is not an approximation of the nearest-neighbour distance — it *is* the nearest-neighbour distance, because the Delaunay triangulation contains the nearest-neighbour graph. It costs O(n) to read off a mesh we already built, with no KD-tree. F1 rose from 0.63 to 0.89.

MAD threshold	Precision (min)	Recall (min)	F1 (min)	F1 (mean)
3.5	0.42	0.96	0.58	0.29
5.0	0.67	0.93	0.77	0.35
6.0	0.76	0.91	0.83	0.38
8.0	0.91	0.89	0.89	0.45
12.0	0.95	0.84	0.88	0.57
14.0	1.00	0.79	0.87	0.63



Left: achieved radius as z, the number of points allowed to go uncovered, increases. Right: detector precision, recall and F1 against the planted ground truth as the threshold varies.

Two results worth saying out loud

- **The last few points are extremely expensive.** Abandoning 20 of 400 points (5%) cuts the radius by about 20%. In a warehouse setting that is the difference between siting a depot for one remote customer and not.

- **The filter looks worse before it looks better, and the crossover is meaningful.** At z = 0 the filtered solution is worse (86.2 vs 60.3) because it discarded points it is then scored on. It overtakes plain Gonzalez at z = 4 and keeps winning — the crossover lands where the filter's own judgement says it should.

Method	Radius (n=250, z=8)	Time	Complexity	Guarantee	Needs z?
Charikar greedy	24.55	3.2 ms	`O(n²k log n)`	3× optimal	yes
Robust Gonzalez	26.57	0.1 ms	`O(nk)`	none	yes
Delaunay filter + Gonzalez	30.96	1.9 ms	`O(n log n + nk)`	none	**no**

Read this table honestly in the review. The method with a proof wins on radius, and our Delaunay filter comes last on this particular measure. Its argument is a different one: it is the only method that is never told how many outliers exist, and it is 30× faster than the one with the proof. If the reviewer asks "so is yours better?", the answer is "it is cheaper and it does not need a parameter nobody actually knows in practice", not "yes".

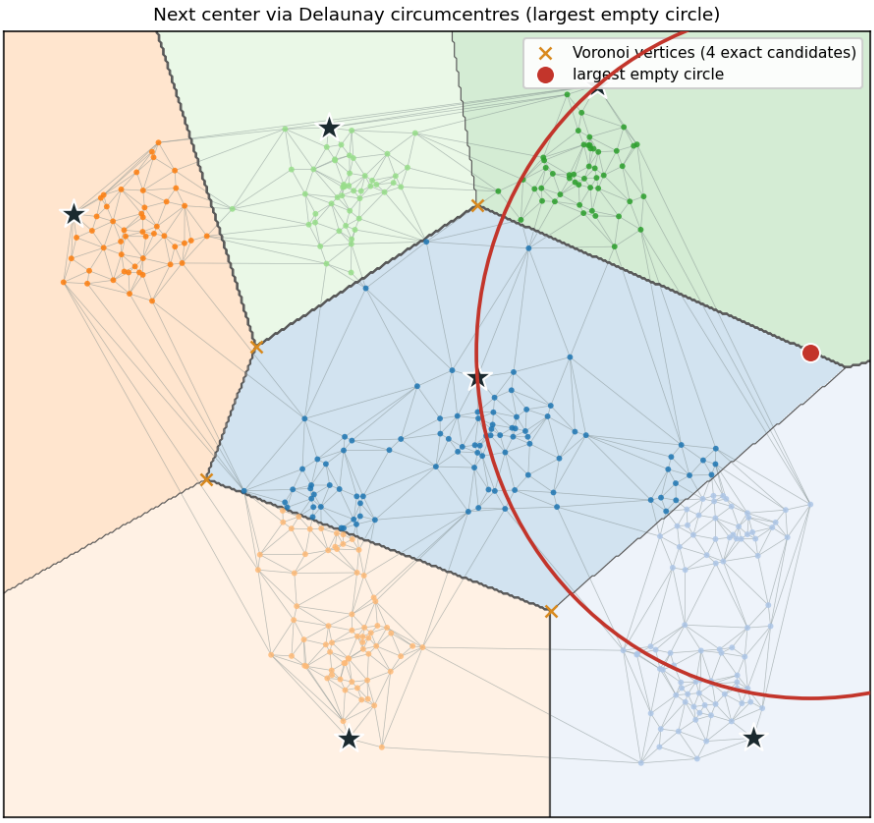
The Delaunay dual as a speed-up for the greedy step

Gonzalez repeatedly needs the point farthest from all current centers. Restricted to the dataset that is a simple O(n) scan. On a **continuous** map it is the classical largest-empty-circle problem, and its answer always sits at a Voronoi vertex of the current centers or on the boundary of the map.

Since Voronoi vertices are exactly the circumcentres of the Delaunay triangles of those centers, the candidate set collapses from a grid to a handful of points.

Approach	Candidates scored	Time	Complexity	Radius found
Grid scan, 500 × 500	250,000	18.3 ms	`O(res²k)`	36.359
Delaunay circumcentres	404	0.3 ms	`O(k log k)`	36.353

619× fewer candidates, 55× faster, agreeing to 99.98%. The remaining 0.02% is honest and worth naming: the interior candidates are exact, but the candidates along the map border are still sampled at 100 points per edge. Computing the exact Voronoi-edge/border intersections would close that gap and is a clean, small piece of further work.



Amber crosses are the exact interior candidates (Delaunay circumcentres). The red circle is the largest empty circle found.

Time complexity of everything in the project

n = points, k = centers, m = restarts, I = k-means iterations, z = outliers allowed, res = grid resolution. Highlighted rows are implemented and measured in this codebase.

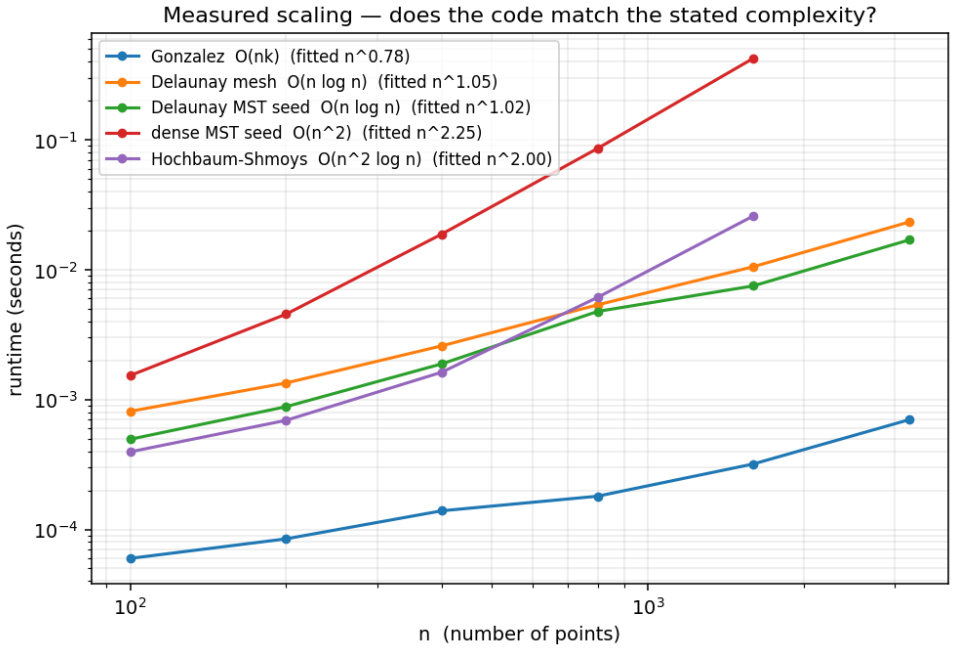
Component	Time complexity	Space	Guarantee	Status
Gonzalez greedy farthest-point	$O(nk)$	$O(n)$	2× optimal	implemented
Gonzalez, step trace for the demo	$O(nk)$	$O(n + k)$	2× optimal	implemented
Multi-start Gonzalez, m restarts	$O(mnk)$	$O(n)$	2× optimal	implemented
Hochbaum–Shmoys parametric pruning	$O(n^2 \log n)$	$O(n^2)$	2× optimal	implemented
Brute-force exact optimum	$O(C(n,k)\cdot nk)$	$O(n^2)$	exact	implemented
Local search swap polish	$O(t\cdot n^2k)$	$O(n)$	none (≤ start)	implemented
Random centers baseline	$O(nk)$	$O(k)$	none	implemented
k-means, Lloyd + k-means++	$O(nkI)$	$O(n + k)$	different objective	implemented
Delaunay triangulation (Qhull)	$O(n \log n)$	$O(n)$	exact	implemented
Delaunay triangulation (browser, Bowyer–Watson)	$O(n^2)$	$O(n)$	exact	implemented
Seed: random / centroid / extreme	$O(1)$ / $O(n)$ / $O(n)$	$O(1)$	—	implemented
Seed: densest via KD-tree	$O(n \log n)$	$O(n)$	—	implemented
Seed: MST centre, dense matrix	$O(n^2)$	$O(n^2)$	—	implemented
Seed: MST centre via Delaunay	$O(n \log n)$	$O(n)$	—	implemented
Seed: largest empty circle	$O(n \log n)$	$O(n)$	—	implemented
Outlier score from mesh (1-NN, MAD)	$O(n)$	$O(n)$	—	implemented
Robust Gonzalez, skip z farthest	$O(nk)$	$O(n)$	none	implemented
Delaunay filter + Gonzalez	$O(n \log n + nk)$	$O(n)$	none	implemented
Charikar outlier greedy	$O(n^2k \log n)$	$O(n^2)$	3× optimal	implemented
Largest empty circle via circumcentres	$O(k \log k)$	$O(k)$	exact interior	implemented
Farthest point by grid scan	$O(res^2\cdot k)$	$O(res^2)$	approximate	implemented
Capacitated k-center, LP rounding	polynomial (LP)	—	9× optimal	not built
Fair k-center (KFC)	polynomial (LP+greedy)	—	3× optimal	not built
Streaming k-center (Guha)	$O(k \log(1/\varepsilon)/\varepsilon)$ memory	sublinear	(2+ε)×	not built

Measured, not just asserted

Stating a complexity is cheap. Experiment 6 times every routine at n = 100…3200 and fits the exponent from a log-log regression, using only n ≥ 400 because below that Python call overhead swamps the actual work.

Routine	Stated	Fitted exponent	Verdict
Gonzalez	`O(nk)`	0.78	below n^1 — memory-bandwidth bound at these sizes
Delaunay mesh	`O(n log n)`	1.05	matches
Delaunay MST seed	`O(n log n)`	1.02	matches
Dense MST seed	`O(n²)`	2.25	matches
Hochbaum–Shmoys	`O(n² log n)`	2.00	matches

The headline is the pair of MST rows: **2.25 versus 1.02**. That is the $O(n^2) \rightarrow O(n \log n)$ claim confirmed by measurement rather than by assertion. A log n factor is invisible to a power-law fit, so an $O(n \log n)$ routine correctly reads as slightly above n^1.



Measured runtime against n, log-log. Parallel lines mean the same growth rate; the gap between the two MST seeding lines widens with n.

Further areas you could add, in order of value per hour

Each of these fits the existing code with no restructuring. The first two are small and already half-built.

Idea	What it involves	Complexity	Effort	Why it is worth it
1. Local-search swap (already coded, under-used)	Replace a center with a nearby point if the radius drops; repeat. The k-center analogue of 2-opt for TSP.	$O(t \cdot n^2 k)$	done — needs writing up	Best empirical result in the project: 1.05× optimal versus 1.54× for plain Gonzalez. Nearly closes the gap to exact.
2. Exact Voronoi/border intersections	Replace the 100-samples-per-edge border scan with real edge-boundary intersection maths.	$O(k \log k)$	an afternoon	Removes the last approximation in the geometry and makes the exactness claim unqualified.
3. Capacity limits	Each center serves at most L points; add a capacity-aware reassignment pass after Gonzalez places centers.	$O(nk)$ heuristic; LP version polynomial, 9×	2–3 days	The most realistic constraint for warehouses, and the theory bound (9×) is so loose that a simple heuristic can look good.
4. Incremental / streaming placement	Add points one at a time and update the triangulation locally instead of rebuilding it.	$O(\log n)$ amortised per insert	2–3 days	Connects directly to the 2026 incrementality result in your research note, which very few student projects will have heard of.
5. Road-network distance	Swap Euclidean distance for shortest-path distance on a real road graph.	$O(nk)$ after an all-pairs preprocess	depends on data	Turns a synthetic demo into a real facility-placement study. Delaunay no longer applies, which is itself an interesting thing to discuss.
6. The Euclidean 1.82 gap	Empirically hunt for instances where Gonzalez approaches 2×, and see whether planar structure caps it lower.	experimental	open-ended	Highest-risk, highest-reward: this is a genuinely open question, not a coursework exercise.

Limitations to state before someone else finds them

Naming these yourself is worth marks. Being caught by them is not.

- **The detector threshold was tuned on the data it was scored on.** The MAD threshold of 8.0 was picked from the same sweep it is then evaluated against. That is optimistic. The fix is small: pick the threshold on one set of generated instances and report on a different set.
- **All data is synthetic.** Planted outliers are drawn uniformly at random, which is the easiest possible case for a density-based detector. Real messy data has clustered errors, duplicates and near-duplicates, and would be harder.
- **The seeding conclusion is a negative result, and negatives need power.** Twelve instances per family is enough to see that the differences are smaller than the spread; it is not enough to prove any two rules are equivalent. A paired Wilcoxon test across ~30 instances would make the claim properly.
- **Multi-start is not cost-matched.** It uses 25× the compute of a single deterministic seed, so of course it wins. The fair comparison gives every method the same time budget.
- **The browser triangulation is O(n²), not O(n log n).** Bowyer–Watson with a naive point-location scan was chosen for readability. It is honest about this in the on-page complexity table; the Python side uses Qhull and is O(n log n).
- **The Delaunay filter loses on radius in the head-to-head.** Its case rests on not needing z and on being 30× faster, not on being more accurate.

How to present this in the review

A structure that puts the strongest material where attention is highest.

Slide	Content	The line to land
1	Problem and the 2× bound	"Unlike TSP there is no headroom to beat — so we explored the edges of the proof instead."
2	One triangulation, three uses (the table on page 1)	`"Everything we added comes from one structure, built once in O(n log n)."`
3	Live simulator: step through the greedy rule	Run it. Change the seed rule. Let them see the centers move.
4	Focus 1 results, with the error bars	"The first point matters by 25–40%, but no cheap rule reliably exploits it — and here is why that is the honest answer."
5	Focus 2: the mean-versus-min decision	"Rim points have long edges but close neighbours. Switching to the minimum took F1 from 0.63 to 0.89."
6	Cost of the last few points	"Dropping 5% of points cuts the radius by 20%."
7	Measured complexity, the 2.25-versus-1.02 fit	"We did not just state the complexity, we measured it."
8	Limitations and next steps	Say the threshold-tuning caveat before you are asked.

Questions you should expect, and short answers

Likely question	Short answer
Why Delaunay rather than just a KD-tree?	A KD-tree answers neighbour queries only. The triangulation additionally gives the MST as a subgraph, and the Voronoi diagram as its dual, so one structure covers seeding, outliers and coverage regions. A KD-tree would need three separate tools.

Likely question	Short answer
Did you beat the 2× bound?	No, and it is proven impossible in general metrics. We measured what is actually achieved: 1.54× on average, worst observed 1.89×, and 1.05× with local search.
Is your outlier method better than Charikar's?	Not on radius. It is 30× faster and needs no z parameter. Charikar has a 3× proof; ours has none.
Why does the filtered solution look worse at z = 0?	Because it discarded points it is then being scored on. It overtakes at z = 4 and the data has 10 planted outliers.
What is the bottleneck at large n?	For plain Gonzalez, nothing much — it is O(nk). The O(n²) routines (Hochbaum–Shmoys, Charikar, dense MST) are what caps us, which is exactly why the Delaunay route matters.

Everything in this document is reproducible from the accompanying archive: `python run_demo.py` regenerates every number and figure, and `web/index.html` opens the simulator with no server and no internet connection.